

Here's the StudentController code:

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.Rendering;
using Microsoft.EntityFrameworkCore;
using System.Linq;
using System.Threading.Tasks;
using UniversityEFCore.Models;
using UniversityEFCore.ViewModels;
using System;

namespace UniversityEFCore.Controllers
{
    public class StudentController : Controller
    {
        private readonly UniversityContext _context;
        private const int PageSize = 5; // adjust pagination size

        public StudentController(UniversityContext context)
        {
            _context = context;
        }

        // GET: /Student/GetAll?page=1&search=Ali&departmentId=2
        public async Task<IActionResult> GetAll(int page = 1, string search
= "", int? departmentId = null)
        {
            var query = _context.Students.Include(s =>
s.Department).AsQueryable();

            if (!string.IsNullOrEmpty(search))
                query = query.Where(s => s.Name.Contains(search));

            if (departmentId.HasValue)
                query = query.Where(s => s.DepartmentId == departmentId);

            int totalCount = await query.CountAsync();
            var students = await query
                .OrderBy(s => s.Name)
                .Skip((page - 1) * PageSize)
                .Take(PageSize)
                .ToListAsync();

            var vm = new StudentListViewModel
            {
                Students = students,
                CurrentPage = page,
                TotalPages = (int)Math.Ceiling(totalCount /
(double)PageSize),
                Search = search,
                DepartmentId = departmentId,
                Departments = new SelectList(await
_context.Departments.ToListAsync(), "Id", "Name")
            };

            return View(vm);
        }

        // GET: /Student/GetById/5
        public async Task<IActionResult> GetById(int id)
        {

```

```

        var student = await _context.Students
            .Include(s => s.Department)
            .FirstOrDefaultAsync(s => s.Id == id);

        if (student == null)
            return NotFound();

        return View(student);
    }

    // GET: /Student/Add
    public async Task<IActionResult> Add()
    {
        ViewBag.Departments = new SelectList(await
            _context.Departments.ToListAsync(), "Id", "Name");
        return View();
    }

    // POST: /Student/Add
    [HttpPost]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> Add(Student student)
    {
        if (ModelState.IsValid)
        {
            _context.Students.Add(student);
            await _context.SaveChangesAsync();
            return RedirectToAction(nameof(GetAll));
        }

        ViewBag.Departments = new SelectList(await
            _context.Departments.ToListAsync(), "Id", "Name", student.DepartmentId);
        return View(student);
    }

    // GET: /Student/Edit/5
    public async Task<IActionResult> Edit(int id)
    {
        var student = await _context.Students.FindAsync(id);
        if (student == null) return NotFound();

        ViewBag.Departments = new SelectList(await
            _context.Departments.ToListAsync(), "Id", "Name", student.DepartmentId);
        return View(student);
    }

    // POST: /Student/Edit/5
    [HttpPost]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> Edit(int id, Student student)
    {
        if (id != student.Id) return BadRequest();

        if (ModelState.IsValid)
        {
            _context.Update(student);
            await _context.SaveChangesAsync();
            return RedirectToAction(nameof(GetAll));
        }

        ViewBag.Departments = new SelectList(await
            _context.Departments.ToListAsync(), "Id", "Name", student.DepartmentId);
        return View(student);
    }

```

```

    }

    // GET: /Student/Delete/5
    public async Task<IActionResult> Delete(int id)
    {
        var student = await _context.Students
            .Include(s => s.Department)
            .FirstOrDefaultAsync(s => s.Id == id);

        if (student == null) return NotFound();

        return View("Warning", student); // Redirect to warning view
    }

    // POST: /Student/DeleteConfirmed/5
    [HttpPost, ActionName("DeleteConfirmed")]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> DeleteConfirmed(int id)
    {
        var student = await _context.Students.FindAsync(id);
        if (student != null)
        {
            _context.Students.Remove(student);
            await _context.SaveChangesAsync();
        }
        return RedirectToAction(nameof(GetAll));
    }
}

```

Supporting ViewModel for Pagination + Filters

```

using Microsoft.AspNetCore.Mvc.Rendering;
using System.Collections.Generic;
using UniversityEFCore.Models;

namespace UniversityEFCore.ViewModels
{
    public class StudentListViewModel
    {
        public IEnumerable<Student> Students { get; set; }
        public int CurrentPage { get; set; }
        public int TotalPages { get; set; }
        public string Search { get; set; }
        public int? DepartmentId { get; set; }
        public SelectList Departments { get; set; }
    }
}

```

✓ Features included:

- **Pagination** with Skip and Take.
 - **Search** by student name.
 - **Filter** by department.
 - **Add/Edit** show department dropdown.
 - **Delete** redirects to a **Warning view** before confirmation.
-

The `async` and `await` keywords in C# are used to make **asynchronous programming** easier and more readable.

Purpose of `async/await`

1. Non-blocking code execution

- Without `async`, if you call a long-running task (like reading from a database, API, or file), the thread will block until it finishes.
- With `await`, the thread is freed to handle other requests while waiting for the result.

2. Better scalability

- In web apps (like ASP.NET Core), `async` allows the server to handle more requests with fewer threads.
- Example: Instead of blocking 100 threads waiting for database results, those threads can be reused for other requests.

3. Cleaner syntax

- Before `async/await`, developers had to use callbacks or `ContinueWith`, which made code messy ("callback hell").
 - `async/await` makes `async` code look like normal synchronous code.
-

How it works

- `async`**: Marks a method as asynchronous and allows the use of `await` inside it.
 - `await`**: Suspends execution of the method until the awaited task completes, then resumes execution without blocking the thread.
-

Example

```
// Without async/await (blocking)
public IActionResult GetData()
{
    var data = GetDataFromDatabase(); // blocks the thread
    return View(data);
}

// With async/await (non-blocking)
public async Task<IActionResult> GetData()
{
    var data = await GetDataFromDatabaseAsync(); // frees thread until
    finished
    return View(data);
}
```

 In short:

- Use `async/await` to write **efficient, scalable, and readable** asynchronous code.
 - It's especially important in **ASP.NET Core** apps, where blocking code reduces the number of concurrent requests the server can handle.
-

1 `AsNoTracking` in Entity Framework Core

- By default, EF Core **tracks** entities it retrieves from the database (so you can later update them and call `SaveChanges()`).
- `AsNoTracking()` tells EF Core **not to track** the returned entities.
- **When to use it:**
 - Read-only queries (e.g., reporting, listing, dropdowns).
 - Improves performance since EF doesn't keep change-tracking metadata.

```
var students = await _context.Students
    .AsNoTracking()
    .ToListAsync();
```

👉 Benefit: Faster queries, less memory used.

👉 Trade-off: You **cannot update** those entities directly unless you attach them again.

2 AutoMapper

- A library that helps map between **objects** automatically (instead of writing repetitive code).
- Example: Converting **Entities (DB models)** to **ViewModels (UI models)**.

Without AutoMapper:

```
var studentVm = new StudentViewModel
{
    Id = student.Id,
    Name = student.Name,
    Age = student.Age
};
```

With AutoMapper:

```
var studentVm = _mapper.Map<StudentViewModel>(student);
```

👉 Benefit:

- Reduces **boilerplate mapping code**.
 - Helps separate **domain models** from **view models**.
-

3 Relative URL

- A **relative URL** points to a resource **relative to the current location** (not the full domain).
- Example:
 - **Absolute URL**: `https://example.com/images/logo.png`
 - **Relative URL**: `/images/logo.png` (relative to the site root) or `../images/logo.png` (relative to the current folder).

👉 In ASP.NET Core Razor:

```
<a href="/student/getall">All Students (absolute path)</a>  
<a href="getall">Relative link from current folder</a>
```

👉 Why use relative URLs?

- Easier to move between environments (dev, test, prod).
- No need to hardcode domain names.

⚡ In short:

- **AsNoTracking** → Faster, read-only queries.
- **AutoMapper** → Maps objects automatically, keeps code clean.
- **Relative URL** → Path based on current location, avoids hardcoding full URLs.